Ellis Architecture

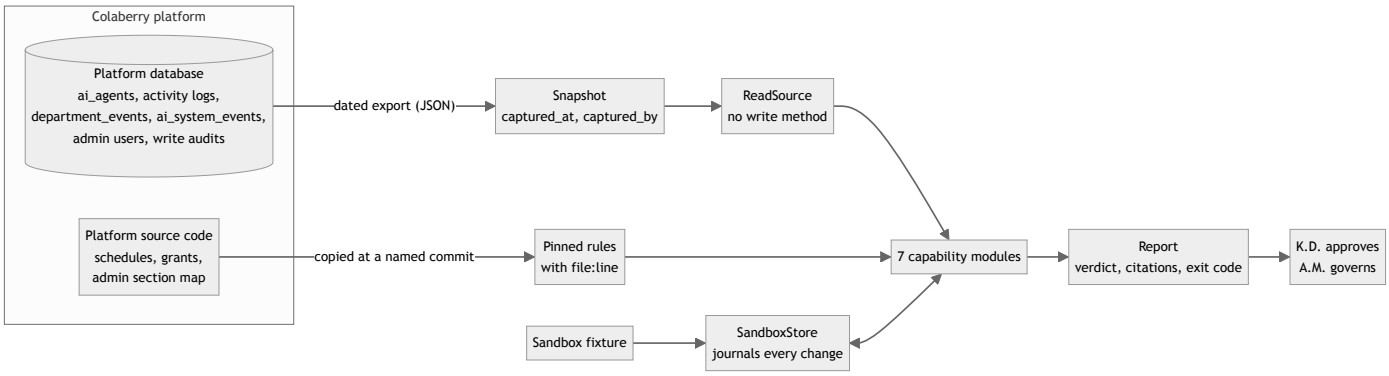
Ellis watches the AI agents that run the Colaberry platform and tells a person, with evidence, what is healthy, what is broken, and what it cannot see. This page shows how it is built and why each part is shaped the way it is.

18 of 48	238	0	0
capabilities built and verified	tests, all passing	runtime dependencies	writes to the live platform

SYSTEM CONTEXT

Ellis reads the platform, never writes to it

The platform is a production system with more than a hundred scheduled agents. Ellis takes two things from it: a dated export of the records it needs, and the rules it needs from the source code, copied at a named commit. Every capability reads through an interface that has no write method. The only thing Ellis ever changes is a sandbox copy.



The export is pulled by a person with admin access. Ellis holds no database credentials and has no network path to the platform.

LAYERS

Five layers, each with one job

L1

Input: dated snapshots

Each capability takes a JSON export with a capture time and who captured it. A snapshot missing a required table is refused, not read as empty.

`src/readSource.ts`
`src/access/accessSnapshot.ts`
`examples/*.json`

L2

Read boundary

Capabilities see the platform only through `ReadSource`, which can find and list rows but has no way to change them. Errors are classified as `SourceUnavailable` or `ContractViolation`.

`src/readSource.ts`
`src/records.ts`

L3

Pinned platform knowledge

Rules Ellis needs from the platform's code are copied with the commit and file:line they came from: agent grants, the admin section map, the scanner list and the retention sweeps. Each copy names its source, so anyone can check it against the platform, and a test pins the copied values so they cannot change unnoticed.

`src/grants/agentGrants.ts`
`src/access/sectionMap.ts`
`src/retention/sweepCatalogue.ts`

L4

Capabilities

One module per capability, each a pure function from snapshot to report. Same rows in any order give the same report, and every suite has a test for it.

`src/capabilities/*.ts`
`src/sandbox/sandboxStore.ts`

L5

Output: reports a person can check

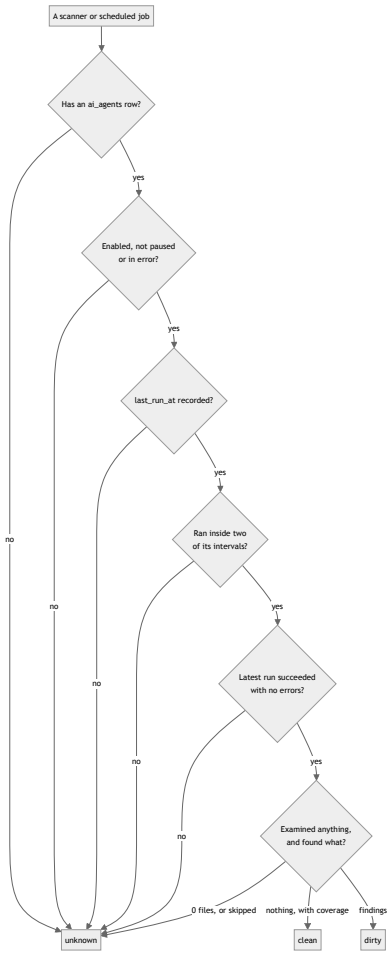
Every verdict carries citations, the table and row id it came from. Each command exits with a code per state, so a scheduler or a person can act on it without parsing text.

`src/cli/*.ts`
`npm run <capability>`

THE CORE RULE

Unknown is checked before clean

The platform's own monitors repeatedly call a silent scanner healthy. Ellis asks every reason a source could be silent before it looks at any result, so "nothing found" and "nothing looked" can never be confused.



● clean ● dirty ● unknown

This is the rule in the security rollup and the security posture scan. The data retention sweep applies the same idea to retention windows: a window that is only in code is unknown, never a number of days.

CAPABILITY REGISTER

What Ellis can answer today

#	CAPABILITY	ANSWER STATES	EXIT CODES	AUTONOMY NOW → CEILING
1	**Orchestration health watch** Is orchestration healthy right now, and when did that last change?	● healthy ● degraded ● critical ● run_failed ● stalled ● not_running ● unknown	0 answer · 2 refused	R2 → R5
2	**Orchestration auto-repair** Do the two approved repairs work on a sandbox, changing each fault once however often they run?	● changed ● no_op ● partial ● failed	0 verified · 1 failed	R4 → R6
3	**Security posture scan** What did the last secret, code and dependency scan find, and when?	● findings ● clean ● unknown	0 known · 3 unknown	R2 → R5

#	CAPABILITY	ANSWER STATES	EXIT CODES	AUTONOMY NOW → CEILING
4	**Runtime threat and access watch** Who can reach which admin section, and what changed since the last check?	no admin row · row, no sections · bridge only · has sections	0 answer	R2 → R5
5	**Agent behaviour and AI-safety watch** Which agents acted outside their grants, and which grant was missing?	finding · unattributable · not emitted	0 none · 4 findings	R2 → R6
6	**Security oversight rollup** Is the platform safe right now, as one verdict over all seven scanners?	green · red · unknown	0 · 3 unknown · 4 red	R2 → R5
7	**Data retention sweeps** What is each table's retention window, and when was it last swept?	stated · conflicting · unknown	0 all known · 3	R2 → R4
8	**Preview stack reaping** Which preview stacks are live, how idle is each, and which are reap candidates?	active · reap candidate · unknown idle	0 all known · 3 unknown or attention	R2 → R4
9	**Stale action recovery** What is stuck right now, for how long, and on which attempt?	ok · stuck · past attempt cap · unknown	0 none · 3 stuck or unknown · 4 past cap	R2 → R4
10	**Agent ticket auto-resolution** Do the ticket auto-closers close only finished work, and do nothing on a second run? (sandbox)	verified · not verified	0 verified · 3 not	R4 → R7
11	**Infrastructure evolution proposals** What has been proposed, why, and what happened to it?	accepted · deferred · rejected · open	0 all decided · 3 any open	R2 → R5
12	**System resilience supervision** How resilient is the platform right now, and how old is each part's last signal?	healthy · degraded · unknown	0 healthy · 3 unknown · 4 degraded	R2 → R5
13	**Agent enable, disable and pause control** Should this agent be stopped, and would pause or disable actually stop it?	disable recommended · no recommendation · pause holds unless a run fails · no row	0 recommended · 3 none	R2 → R2
14	**Workforce technology direction** What tooling does the fleet hold, and where does it diverge from the written direction?	no divergence · divergent · unknown	0 none · 3 unknown · 4 divergent	R2 → R5
15	**Prompt and performance optimisation** Which prompts cost the most, which are slowest, and which have open change proposals?	measured · partial · unknown	0 measured · 3 partial or unknown	R2 → R5
16	**Platform bug fix dispatch** For a reported bug, does a sandboxed baseline test fail for the reported reason before any fix?	baseline confirmed · passes before fix · unreproducible · other reason · did not run	0 confirmed · 4 unreproducible · 3 not	R1 → R4
17	**Agent registry read surface** Is this agent registered, is it on, when does it really run, and what has it done?	registered · disabled · no recorded runs · not registered	0 registered · 3 not or ambiguous	R2 → R2
18	**Cron run instrumentation**	run recorded · no run recorded	0 run recorded · 3 any not	R2 → R2

#	CAPABILITY	ANSWER STATES	EXIT CODES	AUTONOMY NOW → CEILING
	Did this job run, when, how long did it take, and did it fail?	● not instrumented		

Every command also exits 1 for bad input. Autonomy starts at R0 for each capability; the first release reads and reports (R2), except auto-repair and ticket auto-resolution, whose first releases run multi-step tasks on a sandbox (R4), and bug fix dispatch, whose first release only prepares a baseline test (R1). Promotion is one step at a time, on evidence, with K.D.'s approval.

SAFETY BOUNDARIES

What Ellis is built to be unable to do

Change the live platform

Capabilities receive a `ReadSource`. There is no write method to call.

`src/readSource.ts`

Repair anything real

Repairs run only on a `SandboxStore`, which journals every change and refuses writes that change nothing. There is no production target in the code.

`src/sandbox/sandboxStore.ts`

Hold credentials

An access snapshot carrying password, token or secret columns is refused before it is read.

`src/access/accessSnapshot.ts`

Repeat a leaked secret

Security findings pass through a field whitelist. A scanner's stored snippet of the matching line never leaves the module.

`src/capabilities/securityPostureScan.ts`

Police itself

The behaviour watch excludes the watchers' own actions, Ellis included, and counts them separately.

`src/capabilities/agentBehaviorSafetyWatch.ts`

Let a test reach out

Baseline tests run under Node's permission model: they read and write only their own sandbox folder, start no other programs and open no network. The platform checkout is only ever read.

`src/sandbox/baselineRunner.ts`

Stop or start an agent

Ellis says what a disable or pause would and would not stop. It has no call that changes an agent's state; a person does that.

`src/capabilities/agentEnableDisableControl.ts`

Guess

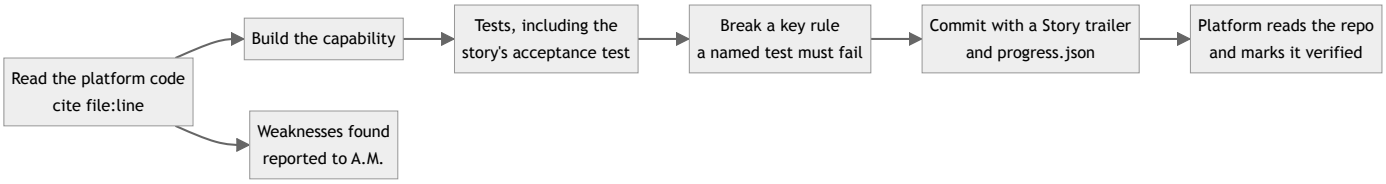
A missing row, run, count or window is reported as unknown or not recorded, with the reason. It is never filled in. A job with no record is "not instrumented", never "did not run".

`every capability`

VERIFICATION

How a capability is proven

Each capability starts from reading the platform code it depends on, and ends with the platform itself confirming the work from this repo. Weaknesses found along the way go to A.M. with file:line references.



The break step is a manual mutation test: change the rule in the code, run the suite, confirm which named test fails, then restore the code. Each capability's write-up records which test caught each break.

ROAD TO PRODUCTION

Dev, then test, then production

NOW · DEV

Proven on tests and example exports

All eighteen capabilities pass their tests and run on example inputs; the bug baseline has also been proven on real platform code. None has been run on live records yet.

NEXT · TEST

First live check

One export from A.M.'s admin screen, with Ellis's answers checked against it together with K.D. K.D. cannot open the admin surfaces today (`requireAdmin`, `aiOpsRoutes.ts:44`).

THEN · PRODUCTION

Scheduled, still read-only

Ellis runs on a schedule against fresh exports. Any action beyond reporting moves up the autonomy ladder one step at a time, with K.D.'s sign-off.

STACK

Small on purpose

TypeScript run natively by Node, with the standard library's test runner. No framework and no runtime packages, so there is nothing to patch and it can all be read in a day: 48 source files, about 5,000 lines, and 238 tests.

`TypeScript 5.8, strict` `Node 23.6+ type stripping` `node:test` `tsc --noEmit` `JSON snapshots` `0 runtime dependencies`

Repository: github.com/qninying/platform-reliability-agent

Platform facts are pinned to the public Colaberry platform repository at the commit named in each capability's write-up, from `c860274` to `c699e58`. State as of 2026-10-27, after capability 18.